

Module 1: Introduction to PL/SQL

Learning Outcomes

At the end of this module, students will be able to:

- Understand the limitations of SQL.
 - Explain the need for PL/SQL.
 - Differentiate between SQL and PL/SQL.
 - Write and execute a basic PL/SQL program.
 - Understand the structure of a PL/SQL block.
 - Use the `DBMS_OUTPUT.PUT_LINE` procedure to display output.
-

1. Introduction to PL/SQL

What is PL/SQL?

PL/SQL (Procedural Language/Structured Query Language) is Oracle Corporation's procedural extension to SQL. It combines the data manipulation capabilities of SQL with the procedural programming features found in languages such as C, Java, and Python.

SQL alone is used to retrieve and manipulate data in relational databases. However, SQL cannot implement programming logic such as loops, conditions, variables, exception handling, or reusable program units. PL/SQL addresses these limitations by allowing developers to write complete programs that execute within the Oracle Database.

In simple terms:

SQL tells the database *what* data to retrieve or modify, whereas PL/SQL tells the database *how* to perform a sequence of operations.

Definition

PL/SQL is Oracle's procedural extension of SQL that enables developers to write complete database programs using variables, conditions, loops, cursors, procedures, functions, triggers, and exception handling.

Why was PL/SQL introduced?

Before PL/SQL, developers had to execute one SQL statement at a time from an application. This resulted in frequent communication between the application and the database server, increasing network traffic and reducing performance.

PL/SQL was introduced to:

- Reduce communication between applications and the database.
 - Improve performance by executing multiple SQL statements together.
 - Provide programming constructs such as variables, loops, and conditional statements.
 - Support modular programming through procedures, functions, and packages.
 - Improve code security and maintainability.
-

Example Without PL/SQL

Suppose we want to:

1. Find an employee's salary.
2. Increase it by 10%.
3. Update the employee table.
4. Display a confirmation message.

Using only SQL, these operations require multiple separate statements:

```
SELECT salary
FROM employee
WHERE emp_id = 101;

UPDATE employee
SET salary = salary * 1.10
WHERE emp_id = 101;

COMMIT;
```

Each statement is sent separately to the database.

Example Using PL/SQL

```
DECLARE
    v_salary NUMBER;
BEGIN
    SELECT salary
    INTO v_salary
```

```
FROM employee
WHERE emp_id = 101;

v_salary := v_salary * 1.10;

UPDATE employee
SET salary = v_salary
WHERE emp_id = 101;

DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully');

COMMIT;
END;
/
```

Here, all the operations are executed as a single PL/SQL block, reducing communication with the database and improving efficiency.

Features of PL/SQL

PL/SQL provides several features that make database programming powerful and efficient:

1. Block-Structured Language

PL/SQL programs are organized into logical blocks, making code modular and easier to understand.

2. Procedural Programming

PL/SQL supports variables, conditions, loops, and reusable program units, enabling developers to implement complex business logic.

3. Tight Integration with SQL

PL/SQL allows SQL statements such as `SELECT`, `INSERT`, `UPDATE`, and `DELETE` to be used directly within procedural code.

4. Exception Handling

Errors can be detected and managed gracefully using exception handling mechanisms.

5. Modular Programming

Business logic can be organized into procedures, functions, packages, and triggers for better maintainability.

6. High Performance

Executing multiple SQL statements as a single PL/SQL block reduces network traffic and improves execution speed.

7. Portability

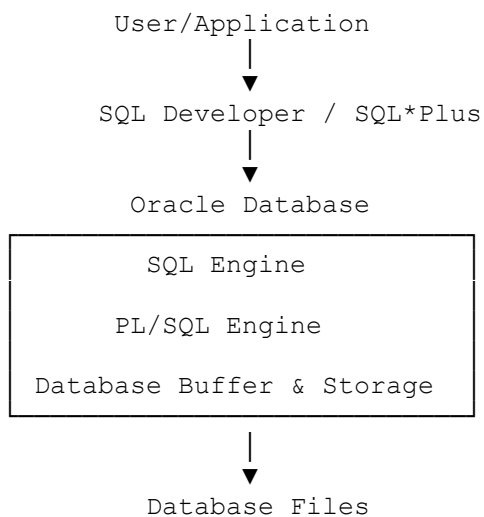
PL/SQL programs can run on any Oracle Database platform with minimal or no changes.

Advantages of PL/SQL

- Reduced network traffic.
 - Faster execution.
 - Better security through stored program units.
 - Improved maintainability.
 - Reusable code.
 - Better error handling.
 - Support for complex business logic.
-

Architecture of Oracle Database

Understanding where PL/SQL executes is essential.



Working

1. The user submits a PL/SQL block.
2. The PL/SQL Engine processes the procedural statements.
3. Whenever an SQL statement is encountered, it is passed to the SQL Engine.
4. The SQL Engine executes the SQL statement.
5. Results are returned to the PL/SQL Engine.
6. The PL/SQL Engine completes execution and returns the final output.

SQL vs PL/SQL

Feature	SQL	PL/SQL
Language Type	Declarative	Procedural
Execution	One statement at a time	Executes an entire block
Variables	Not supported	Supported
Loops	Not supported	Supported
Conditional Statements	Not supported	Supported
Exception Handling	Not supported	Supported
Cursors	Limited	Fully supported
Program Units	No	Procedures, Functions, Packages, Triggers
Performance	Lower for multiple operations	Better for complex operations

Structure of a PL/SQL Block

Every PL/SQL program consists of four sections.

```
DECLARE
    -- Variable declarations

BEGIN
    -- Executable statements

EXCEPTION
    -- Error handling

END;
```

1. DECLARE Section (Optional)

Used to declare variables, constants, cursors, and user-defined exceptions.

Example:

```
DECLARE
```

```
v_name VARCHAR2(50);  
v_salary NUMBER;
```

2. BEGIN Section (Mandatory)

Contains executable statements.

Example:

```
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL');  
END;  
/
```

3. EXCEPTION Section (Optional)

Handles runtime errors.

Example:

```
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('An error occurred.');
```

4. END Section (Mandatory)

Marks the end of the PL/SQL block.

```
END;  
/
```

The / tells SQL*Plus or SQL Developer to execute the PL/SQL block.

Your First PL/SQL Program

```
SET SERVEROUTPUT ON;  
  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL Programming');  
END;  
/
```

Output

Understanding `DBMS_OUTPUT.PUT_LINE`

`DBMS_OUTPUT` is an Oracle-supplied package that enables PL/SQL programs to display messages to the user.

Syntax

```
DBMS_OUTPUT.PUT_LINE('Message');
```

Example

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello Students');
    DBMS_OUTPUT.PUT_LINE('Welcome to Oracle PL/SQL');
END;
/
```

Output

```
Hello Students
Welcome to Oracle PL/SQL
```

Real-World Applications of PL/SQL

PL/SQL is widely used in enterprise applications, including:

- Banking Systems
 - Hospital Management Systems
 - Payroll Processing
 - Inventory Management
 - University Information Systems
 - ERP Applications
 - E-commerce Platforms
-

Summary

In this module, we learned that PL/SQL extends SQL by introducing procedural programming capabilities. We explored its features, advantages, architecture, differences from SQL, the structure of a PL/SQL block, and wrote our first PL/SQL program using

`DBMS_OUTPUT.PUT_LINE`.

Practice Exercises

1. Write a PL/SQL program to print your name, branch, and university.
2. Display the message: **"Welcome to Oracle Database Programming."**
3. Print three separate lines using `DBMS_OUTPUT.PUT_LINE`.
4. Create a PL/SQL block that displays your favorite programming language.
5. Modify the first PL/SQL program to print the current academic year.

Viva/Interview Questions

1. What is PL/SQL?
2. Why was PL/SQL introduced?
3. What are the limitations of SQL?
4. Explain the structure of a PL/SQL block.
5. What is the purpose of the `DECLARE` section?
6. Why is the `BEGIN` section mandatory?
7. What is the role of the `EXCEPTION` section?
8. Why is `DBMS_OUTPUT.PUT_LINE` used?
9. Differentiate between SQL and PL/SQL.
10. List five real-world applications of PL/SQL.